

On-Device AI Starts Here

Start developing with Qualcomm® AI Hub

Qualcomm AI Hub is the place to optimize, validate, and deploy any model in a few lines of code for Qualcomm devices. Our four products can be used independently or together as part of your on-device development workflow.

Follow the walkthrough that best fits your needs:

-  **Getting started with Qualcomm® AI Hub GenieX.** Run frontier language and vision-language models locally on Hexagon NPU, Adreno GPU, or CPU with a few lines of code, using the llama.cpp or QAI RT plugin that best fits your deployment.
-  **Getting started with Qualcomm® AI Hub Models.** Browse and download pre-optimized models covering vision, speech, audio, and text applications, validated on Qualcomm® chipsets.
-  **Getting started with Qualcomm® AI Hub Apps.** Download and customize sample application code with a model already bundled, ready to deploy on-device. Browse the catalog from the qai-hub-apps CLI or filter by domain, operating system, and more.
-  **Getting started with Qualcomm® AI Hub Workbench.** Optimize your custom trained model for a target runtime, run inference, and profile on-device performance with a few lines of code. Target 50+ hosted Qualcomm devices, provisioned within minutes.

Start

Qualcomm AI Hub

Models

Your own model

Choose your starting point — a custom trained model or one from our collection and kick off your workflow.



Qualcomm AI Hub

Workbench

compile | run inference | quantize

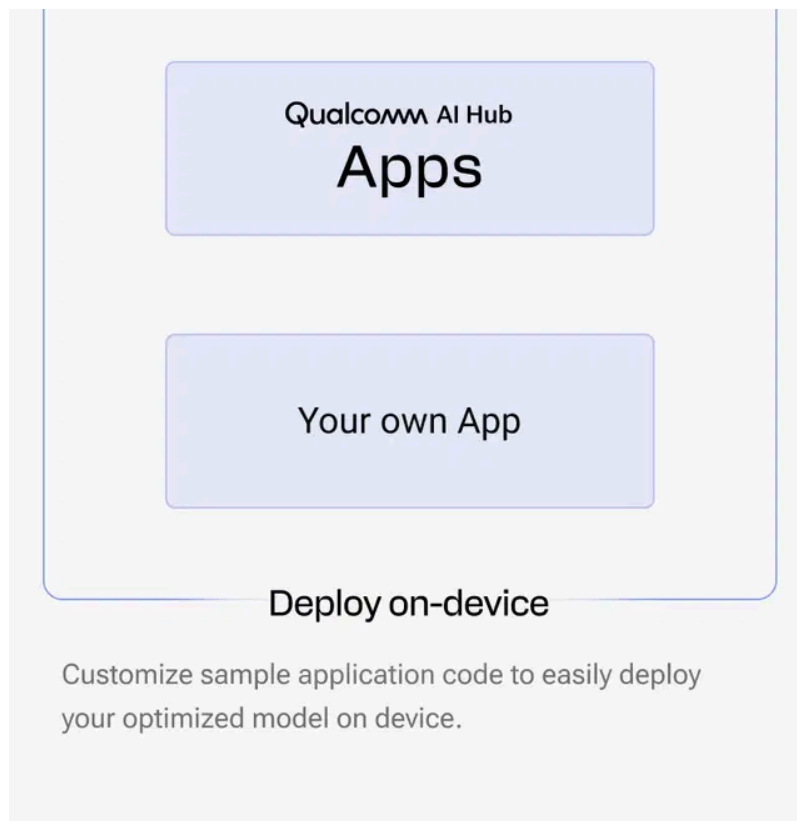
Qualcomm AI Hub

GenieX

NEW

Easily compile, run inference and quantize your model to optimize for edge devices.





[Account Setup](#) [GenieX](#) [Models](#) [Apps](#) **[Workbench](#)**

Getting Started with AI Hub Workbench

Optimize a trained model for your device. Bring an open source or custom trained model to Workbench and specify your target device and runtime for automated conversion and translation into a model ready for efficient on-device deployment.



Workbench can be used to quantize models and run inference on cloud-hosted Qualcomm devices for easy experimentation, allowing you to iteratively validate and profile to reach the ideal balance of speed, accuracy, and memory usage for your use case.

In this flow, we are optimizing the open source MobileNet model for the Samsung Galaxy S25 (Family), which uses a TFLite runtime. If you haven't already, create a Workbench account following the instructions in [Account Setup](#) before diving in.

1. Set up dependencies

Open your preferred Python IDE and install any packages needed. For the MobileNet model, you can install the dependencies from AI Hub Models with this line:

```
pip3 install 'qai-hub[torch]'
```



2. Import and export model

Workbench requires an exported model for compilation. We can load this pre-trained MobileNet model, put it into eval mode, and export it with `torch.export.export`. If you are bringing a custom trained model that has already been exported, you can skip the export step!

```
# Import torch and pre-trained MobileNet
import torch
from torchvision.models import mobilenet_v2, MobileNet_V2_Weights
torch_model = mobilenet_v2(weights=MobileNet_V2_Weights.IMAGENET1K_V1)
torch_model.eval()

# Export model
input_shape = (1, 3, 224, 224)
example_input = torch.rand(input_shape)
with torch.no_grad():
    exported_torch_model = torch.export.export(torch_model, (example_input,))
```



3. Select device and compile

Import Workbench, set your target device, and compile the model for TFLite. Behind the scenes, Workbench optimizes the model to work with the Qualcomm AI Stack. Follow the compile job link to access the output model, parameters, logs, and more. All jobs are all saved to your Workbench dashboard for future reference.

```
import qai_hub as hub
device = hub.Device("Samsung Galaxy S25 (Family)")

# Compile model
compile_job = hub.submit_compile_job(
    model=exported_torch_model,
```



```
        device=device,
        input_specs=dict(image=input_shape),
        options="--target_runtime tflite"
    )
    target_model = compile_job.get_target_model()
```

4. Submit an inference job

Test your model on device. We'll set up the input image and run on a real cloud-hosted device — be patient with the provisioning and setup! You can click into the inference job link to see the minimum inference time and estimated peak memory usage through the Workbench web UI.

```
import requests
import numpy as np
from PIL import Image

# Provide sample image
sample_image_url = (
    "https://qaihub-public-assets.s3.us-west-2.amazonaws.com/apidoc/input_image1"
)

# Convert image to numpy array to match input specs
response = requests.get(sample_image_url, stream=True)
response.raw.decode_content = True
image = Image.open(response.raw).resize((224, 224))
input_array = np.expand_dims(
    np.transpose(np.array(image, dtype=np.float32) / 255.0, (2, 0, 1)), axis=0
)

# Run inference using input image
inference_job = hub.submit_inference_job(
    model=target_model,
    device=device,
    inputs=dict(image=[input_array]),
)
on_device_output = inference_job.download_output_data()
```

5. Validate model accuracy

Post-process the on-device output to check the results against the [input image](#).

```
# Post-process the on-device output
output_name = list(on_device_output.keys())[0]
out = on_device_output[output_name][0]
on_device_probabilities = np.exp(out) / np.sum(np.exp(out), axis=1)

# Read the class labels for imagenet
```

```

sample_classes = "https://qaihub-public-assets.s3.us-west-2.amazonaws.com/apidoc/
response = requests.get(sample_classes, stream=True)
response.raw.decode_content = True
categories = [s.decode().strip() for s in response.raw]

# Display input image
from IPython import import display
display.Image(sample_image_url)

# Print top 5 predictions for the on-device model
print("Top-5 On-Device predictions:")
top5_classes = np.argsort(on_device_probabilities[0], axis=0)[-5:]
for c in reversed(top5_classes):
    print(f"{c} {categories[c]:20s} {on_device_probabilities[0][c]:>6.1%}")

```

6. Profile on-device

The inference predictions seem reasonable, so let's move on to checking how the model performs on the device with a profile job. Follow the profile job link to double check that your model is using the expected compute unit (in this case, the NPU) and explore detailed metrics such as

analysis, and more.

```

# Submit profile job
profile_job = hub.submit_profile_job(
    model=target_model,
    device=device,
)

```



Profile Job Results

[Jobs](#) > [jjg1xw75](#) Results Ready
SHARE VISUALIZE

Information

Name

efficientnet_b0_float

Target Device

Samsung Galaxy S25 (Family)

Android 15

Snapdragon® 8 Elite for Galaxy | SM8750-AC

Creator

Target Model

job_jpvh4397p_optimized_tflite mqe2l0z7q

Input Specs

image_tensor : float32[1, 224, 224, 3]

Submission / Completion Time

11/14/2025, 11:58:40 AM

11/14/2025, 12:01:29 PM

Versions

TensorFlow Lite : 2.17.0

QAI RT : v2.39.0.250925215840_163802

QNN TFLite Delegate : v2.39.0.250925215840_163802

Android : 15 (AP3A.240905.015.A2)

AI Hub : aihub-2025.11.03.0

Inference Metrics

Minimum Inference Time

0.8 ms

Estimated Peak Memory Usage

0 - 36 MB

Compute Units

NPU 245



Unhappy with the model accuracy? Consider trying a different precision, re-training your model, or starting from the beginning, with a different open source model.

7. (Optional) Quantize your model

Do your profile job results show too much memory consumption? If you're using an open source model, consider quantizing your model to make it smaller.

- a. **Compile to ONNX.** Workbench quantize jobs require a compiled ONNX model as an input. Even if your model is already in ONNX format, compiling this way will ensure optimized patterns for quantization.

```
# Compile your model to ONNX
compile_job = hub.submit_compile_job(
    model=exported_torch_model,
    device=device,
    input_specs=dict(image=input_shape),
    options="--target_runtime onnx"
)
unquantized_onnx_model = compile_job.get_target_model()
```



- b. **Load and pre-process calibration data.** For this example, we'll use 100 sample images from the PyTorch Imagenette dataset as our calibration data. Generally, we recommend using 500-1000 samples. Before running the code below, first download [imagenette_samples.zip](#) and unzip it in your local directory.

```
# Load and pre-process calibration data
# This transform is required for PyTorch ImageNet classifiers
# Source: https://pytorch.org/hub/pytorch_vision_resnet/
import os

mean = np.array([0.485, 0.456, 0.406]).reshape((3, 1, 1))
std = np.array([0.229, 0.224, 0.225]).reshape((3, 1, 1))
sample_inputs = []

images_dir = "imagenette_samples/images"

for image_path in os.listdir(images_dir):
    image = Image.open(os.path.join(images_dir, image_path))
    image = image.convert("RGB").resize(input_shape[2:])
    sample_input = np.array(image).astype(np.float32) / 255.0
    sample_input = np.expand_dims(np.transpose(sample_input, (2, 0, 1)), 0)
    sample_inputs.append(((sample_input - mean) / std).astype(np.float32))
calibration_data = dict(image=sample_inputs)
```



c. **Submit a quantize job.** Now we're ready to run the quantize job, in this case quantizing to INT8.

```
# Submit quantize job
quantize_job = hub.submit_quantize_job(
    model=unquantized_onnx_model,
    calibration_data=calibration_data,
    weights_dtype=hub.QuantizeDtype.INT8,
    activations_dtype=hub.QuantizeDtype.INT8,
)
quantized_onnx_model = quantize_job.get_target_model()
```



d. **Compile to your target runtime.** Quantized models can be compiled to any runtime; compile for TFLite runtime to support your mobile device.

```
# Optimize model for the chosen device
compile_job = hub.submit_compile_job(
    model=quantized_onnx_model,
    device=device,
    input_specs=dict(image=input_shape),
    options="--target_runtime tflite"
)
target_model = compile_job.get_target_model()
```



e. **Validate accuracy and performance.** Evaluate this quantized model for accuracy and performance by repeating steps 4-6 (submitting an inference job, checking accuracy, and submitting a profile job) from above.

If the accuracy or performance is not meeting your needs, you can also re-quantize the model at a [higher precision](#). If you are bringing a custom-trained model, we suggest quantizing with your own compute resources using Qualcomm's PyTorch ONNX [AI Model Efficiency Toolkit \(AIMET\)](#) prior to running a Workbench compile job to optimize for your device.

8. Download optimized model

If everything looks good, download your freshly optimized model to your machine.

```
target_model.download("mobilenet_v2.tflite")
```



Extra Credit: Want to bundle this model into an app to deploy to your Samsung Galaxy S25 device? Keep going with the [Apps section](#)!



Note: Large language models (LLMs) are, by definition, large — meaning that they require extra considerations when being deployed on memory-constrained devices. If you're working with a LLM, we recommend following the [GenieX Docs](#).



[Terms of Use](#) [Privacy Policy](#) [Cookie Policy](#) [Contact Us](#) [Cookie Settings](#)

© 2026 Qualcomm Technologies, Inc. and/or its affiliated companies.

References to "Qualcomm" may mean Qualcomm Incorporated, or subsidiaries or business units within the Qualcomm corporate structure, as applicable.

Qualcomm Incorporated includes our licensing business, QTL, and the vast majority of our patent portfolio. Qualcomm Technologies, Inc., a subsidiary of Qualcomm Incorporated, operates, along with its subsidiaries, substantially all of our engineering, research and development functions, and substantially all of our products and services businesses, including our QCT semiconductor business.

Materials that are as of a specific date, including but not limited to press releases, presentations, blog posts and webcasts, may have been superseded by subsequent events or disclosures.

Nothing in these materials is an offer to sell any of the components or devices referenced herein.